



**DSA – Sec A**

**Lab – 07**

|                               |                                                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b><u>Instructor</u></b>      | <b><u>Mr. Shehryar</u></b>                                                                                   |
| <b><u>Lab Instructors</u></b> | <ul style="list-style-type: none"><li>• Rimsha Aamir</li><li>• Adeel Zafar</li><li>• Muhammad Taha</li></ul> |

**Department of Computer Science, ITU, Lahore.**

## Tasks:

### 1. Implement Custom Iterator for Singly Linked List

- Modify your custom singly linked list to support its own iterator. Implement the following iterator methods:
  - `begin()` : Returns an iterator pointing to the first node.
  - `operator++()` : Move the iterator to the next node.
- Use this iterator in a simple `for` loop to traverse and print the list.

### 2. Use recursion to print Linklist

### 3. Use recursion to print LinkList in reversed order.

### 4. Insert Element at Nth Position using Iterator (Custom Singly Linked List)

- Modify your existing singly linked list implementation to add a function `insertAtNth(...)` using **Iterator**, that inserts a new node with the given data at the specified position.
- Handle cases where the position is out of bounds (greater than the length of the list).

### 5. Reverse List in a Given Range Using Custom Iterator

- Add a function `reverseInRange(iterator start, iterator end)` to reverse the portion of the list between the given `start` and `end` iterators (inclusive).
- You will need to carefully manipulate pointers within your singly linked list, ensuring that only the portion of the list between the two iterators is reversed.

### 6. Merge Two Singly Linked Lists (Custom Singly Linked List)

- Implement a function `mergeLists(SinglyLinkedList& list2)` that merges another singly linked list (`list2`) into the current list. Ensure the merged list maintains the correct order.

### 7. Reverse List Using Iteration (STL `forward_list`)

- Using the STL `forward_list`, write a function `reverseList()` that reverses the list iteratively.
- You must use iterators provided by the STL to traverse the list.

### 8. Find Nth Element from End (Custom Singly Linked List)

- Implement a function `findNthFromEnd(int n)` to find and return the `n`th element from the end of the list in a single traversal.

#### 9. Compare Two Lists for Equality (STL `forward_list`)

- Implement a function `areListsEqual (forward_list<int>& list1, forward_list<int>& list2)` that checks if two `forward_list` objects contain the same elements in the same order.
- Use iterators to traverse and compare both lists.

#### 10. Sort a Singly Linked List (Custom Singly Linked List)

- Add a function `sortList ()` to your singly linked list implementation that sorts the elements in ascending order. You can choose any sorting algorithm (e.g., bubble sort or selection sort).